

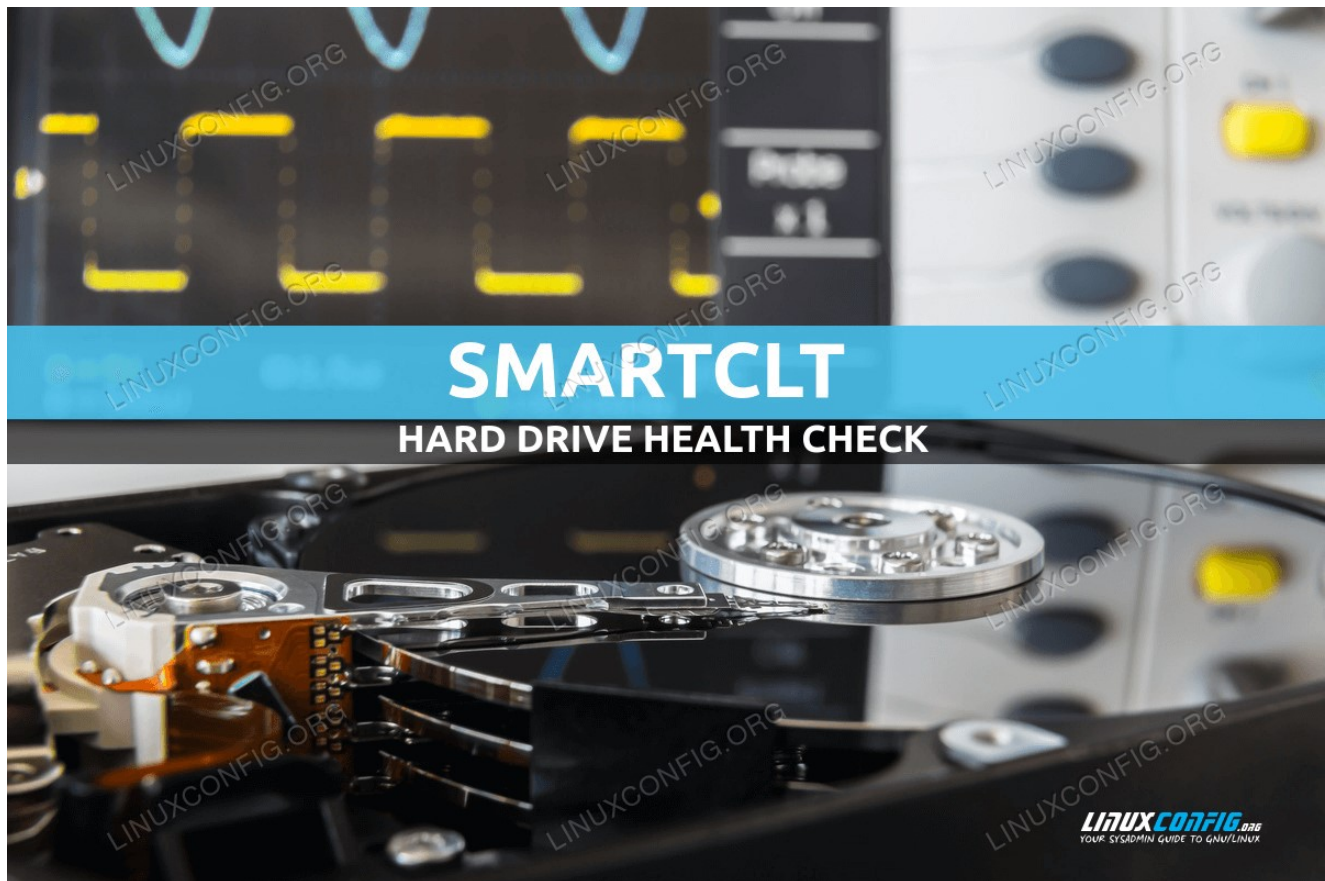
How to check an hard drive health from the command line using smartctl

26 September 2020 by Egidio Docile

The **smartmontools** package is generally available in the default repositories of all the major Linux distributions. It contains two utilities useful to check the status of storage with **S.M.A.R.T** support (*Self Monitoring Analysis and Reporting Technology*): **smartctl** and **smartd**. The former is the utility we use directly to check S.M.A.R.T attributes, run tests, or perform other actions; the latter is the daemon which can be used to schedule operations in the background. In this tutorial we will learn the basic usage of **smartctl**.

In this tutorial you will learn:

- How to install smartmontools package on various distributions
- What are the differences between the S.M.A.R.T self-tests
- How to use smartctl to check the health of a storage device
- How to run tests on a storage device from the command line



How to check an hard drive health from the command line using smartctl

Software requirements and conventions used

Software Requirements and Linux Command Line Conventions

Category	Requirements, Conventions or Software Version Used
System	Distribution independent
Software	The smartmontools package (see instructions)
Other	Root permissions
Conventions	# – requires given linux-commands to be executed with root privileges either directly as a root user or by use of sudo command \$ – requires given linux-commands to be executed as a regular non-privileged user

Installation

As mentioned before the **smartmontools** package is available in the repositories of all the major Linux distributions, therefore all we have to do to install it, is to

use our favorite package manager. If you are running on Debian or one of its derivatives, like Ubuntu or Mint, for example, you can run:

```
$ sudo apt-get update && sudo apt-get install smartmontools
```

On recent versions of Red Hat Enterprise Linux, CentOS and Fedora we can use **dnf**:

```
$ sudo dnf install smartmontools
```

If Archlinux is your favorite distribution, you can use **pacman**:

```
$ sudo pacman -S smartmontools
```

Checking if SMART is enabled

Let's become familiar with the **smartctl** utility. The first thing we want to check is if S.M.A.R.T support is active on the device. To perform this operation we can run the smartctl utility with the **-i** option (short for **--info**):

```
$ sudo smartctl -i /dev/sda
```

The output of the command is the following:

```
=== START OF INFORMATION SECTION ===
Model Family:      Western Digital Red
Device Model:      WDC WD10EFRX-68FYTN0
LU WWN Device Id:  5 0014ee 20c672def
Firmware Version:  82.00A82
User Capacity:     1,000,204,886,016 bytes [1.00 TB]
Sector Sizes:      512 bytes logical, 4096 bytes physical
Rotation Rate:     5400 rpm
Device is:         In smartctl database [for details use: -P show]
ATA Version is:    ACS-2 (minor revision not indicated)
SATA Version is:   SATA 3.0, 6.0 Gb/s (current: 3.0 Gb/s)
Local Time is:     Thu Sep 24 18:13:19 2020 CEST
SMART support is:  Available - device has SMART capability.
SMART support is:  Disabled
```

We can see that basic information are displayed such as the device family, model, sector sizes, etc. What interests us the most, however is the content of the last two lines. From there we can see that the device has SMART capabilities and that, in this case, SMART support is disabled. What if we want to enable it? All we have to do is to run **smartctl** with the **-s** option, using “on” as argument:

```
$ sudo smartctl -s on /dev/sda
smartctl 6.6 2017-11-05 r4594 [armv6l-linux-5.4.51+] (local build)
Copyright (C) 2002-17, Bruce Allen, Christian Franke, www.smartmontool

=== START OF ENABLE/DISABLE COMMANDS SECTION ===
SMART Enabled.
```

Getting familiar with smartctl

To get all the available SMART information about a storage device, we can launch the utility with the **-a** option (short for **-all**) and of course pass the path of the device we want to check as the last argument of the command. Suppose we want to check the current status of the **/dev/sda** device; we would run:

```
$ sudo smartctl -a /dev/sda
```

The command above produces a lot of output. Among the other things, we can see the status of various SMART parameters:

```
SMART Attributes Data Structure revision number: 16
Vendor Specific SMART Attributes with Thresholds:
ID# ATTRIBUTE_NAME          FLAG     VALUE WORST THRESH TYPE      UPDA
  1 Raw_Read_Error_Rate     0x002f   200    200   051    Pre-fail  Alwa
  3 Spin_Up_Time            0x0027   135    125   021    Pre-fail  Alwa
  4 Start_Stop_Count        0x0032   100    100    00     Old_age  Alwa
  5 Reallocated_Sector_Ct   0x0033   200    200   140    Pre-fail  Alwa
  7 Seek_Error_Rate         0x002e   200    200    00     Old_age  Alwa
  9 Power_On_Hours          0x0032   085    085    00     Old_age  Alwa
 10 Spin_Retry_Count        0x0032   100    100    00     Old_age  Alwa
 11 Calibration_Retry_Count 0x0032   100    100    00     Old_age  Alwa
 12 Power_Cycle_Count       0x0032   100    100    00     Old_age  Alwa
192 Power-Off_Retract_Count 0x0032   200    200    00     Old_age  Alwa
193 Load_Cycle_Count       0x0032   199    199    00     Old_age  Alwa
194 Temperature_Celsius    0x0022   111    099    00     Old_age  Alwa
196 Reallocated_Event_Count 0x0032   200    200    00     Old_age  Alwa
197 Current_Pending_Sector  0x0032   200    200    00     Old_age  Alwa
198 Offline_Uncorrectable   0x0030   100    253    00     Old_age  Offl
199 UDMA_CRC_Error_Count    0x0032   200    200    00     Old_age  Alwa
200 Multi_Zone_Error_Rate   0x0008   200    200    00     Old_age  Offl
```

Very important parameters to check are, among the others, “Reallocated_Sector_Ct” and “Current_Pending_Sector”. In both cases if the **RAW_VALUE** is something other than 0, we should be very careful and start to backup data on the hard drive. The **Reallocated_Sector_Ct** is the count of sectors on the block device which cannot be used correctly.

When such a sector is found it is remapped to one of the available spare sectors of the storage device, and data contained in it is relocated. The **Current_Pending_Sector** attribute, instead, is the count of bad sectors that are still waiting to be remapped. If you want to know more about the S.M.A.R.T attributes and their meaning, you can begin to take a look at the [wikipedia S.M.A.R.T page](#).

In the output we can also see a log of the tests performed on the device:

```
SMART Self-test log structure revision number 1
Num  Test_Description      Status                Remaining  LifeTime(h)
# 1  Short offline          Completed without error    00%         9590
# 2  Short offline          Completed without error    00%         2941
# 3  Extended offline       Completed without error    00%          21
# 4  Short offline          Completed without error    00%          18
```

# 5	Short offline	Completed without error	00%	0
# 6	Short offline	Completed without error	00%	0

In the **Test_Description** column, we can see various kind of tests were run, and all of them were completed without error. In the next section we will see what are the differences between them and how to actually launch a test on a storage device.

Available SMART tests

The **smartctl** utility can be used to launch a variety of self-tests:

- short
- long
- conveyance (ATA devices only)
- select (ATA devices only)

Let's quickly see what are the differences between them.

The **short** test is meant to quickly check the most common problems that could be found on a storage device. The test should take no more than 10 minutes: mechanical, electrical and read performances of a disk are checked.

The **long** test is basically a more accurate version of the “short” test. It can take a lot of time to complete: as stated in the the smartctl manual, it can last from tens of minutes to several hours.

The **conveyance** test is meant to check for possible damages occurred during the transportation of the device. It usually takes minutes to complete a conveyance test. It is available only on ATA devices.

The **select** test, like the “conveyance” one, is available only on ATA devices, and is meant to check only the specified range of LBAs (Logical Block Addresses). The range of addresses is specified when launching the test. For example, to check addresses from 10 to 20 (inclusive), we would run:

```
$ sudo smartctl -t select,10-20
```

It is possible to specify a maximum of 5 different ranges of LBAs to check by repeating the **-t** option:

```
$ sudo smartctl -t select,0-5 -t select,5-10
```

The **-t** option is the short for **--test** and is used to execute a test immediately.

Running a test

We saw what are the possible tests we can run with the **smartctl** utility. Now let's see how to actually launch a test. As we saw in the end of the previous section, the **-t** option is used to run a test immediately; we must provide the type of test we want to run as argument of the option. To execute a **short** test on the **/dev/sda** device we would run:

```
$ sudo smartctl -t short /dev/sda
smartctl 6.6 2017-11-05 r4594 [armv6l-linux-5.4.51+] (local build)
Copyright (C) 2002-17, Bruce Allen, Christian Franke, www.smartmontool

=== START OF OFFLINE IMMEDIATE AND SELF-TEST SECTION ===
```

```
Sending command: "Execute SMART Short self-test routine immediately in
Drive command "Execute SMART Short self-test routine immediately in of
Testing has begun.
Please wait 2 minutes for test to complete.
Test will complete after Thu Sep 24 14:39:05 2020

Use smartctl -X to abort test.
```

The output of the command reports the time we should wait for the test to finish and the date and time when it should be complete. After the specified time interval, to check the results of the test we can run:

```
$ sudo smartctl -a /dev/sda
```

As you can notice the test (The first in the list – #1) and its results have been added to the log list. It was completed without errors:

```
SMART Self-test log structure revision number 1
Num Test_Description Status Remaining LifeTime(h)
# 1 Short offline Completed without error 00% 11286
# 2 Short offline Completed without error 00% 9590
# 3 Short offline Completed without error 00% 2941
# 4 Extended offline Completed without error 00% 21
# 5 Short offline Completed without error 00% 18
# 6 Short offline Completed without error 00% 0
# 7 Short offline Completed without error 00% 0
```

It is possible to know the estimated time a test would take to finish. Such information should be included in the output of the `smartctl -a /dev/sdx` command, but can be requested explicitly by launching `smartctl` with the `-c` option (short for `--capabilities`). The following are the interesting lines in the output:

```
$ sudo smartctl -c /dev/sda
[...]
Short self-test routine
recommended polling time: ( 2) minutes.
Extended self-test routine
recommended polling time: (157) minutes.
Conveyance self-test routine
recommended polling time: ( 5) minutes.
[...]
```


Let's run a conveyance test, now:

```
$ sudo smartctl -t conveyance /dev/sda
```

We wait 5 minutes, and then check the results. As expected the test now appears in the list, and luckily no errors were found:

```
SMART Self-test log structure revision number 1
Num  Test_Description      Status              Remaining  LifeTime(h)
# 1  Conveyance offline    Completed without error    00%        11286
# 2  Short offline        Completed without error    00%        11286
# 3  Short offline        Completed without error    00%        9590
# 4  Short offline        Completed without error    00%        2941
# 5  Extended offline     Completed without error    00%         21
# 6  Short offline        Completed without error    00%         18
# 7  Short offline        Completed without error    00%          0
# 8  Short offline        Completed without error    00%          0
```

Now, for a simple **select** test:

```
$ sudo smartctl -t select,100-150 /dev/sda
smartctl 6.6 2017-11-05 r4594 [armv6l-linux-5.4.51+] (local build)
Copyright (C) 2002-17, Bruce Allen, Christian Franke, www.smartmontool

=== START OF OFFLINE IMMEDIATE AND SELF-TEST SECTION ===
Sending command: "Execute SMART Selective self-test routine immediatel
```

```

SPAN          STARTING_LBA          ENDING_LBA
0              100                    150
Drive command "Execute SMART Selective self-test routine immediately i
Testing has begun.

```

This test is successfully completed:

```

SMART Self-test log structure revision number 1
Num  Test_Description      Status              Remaining  LifeTime(h)
# 1  Selective offline       Completed without error    00%        11287
# 2  Conveyance offline      Completed without error    00%        11286
# 3  Short offline          Completed without error    00%        11286
# 4  Short offline          Completed without error    00%        9590
# 5  Short offline          Completed without error    00%        2941
# 6  Extended offline       Completed without error    00%         21
# 7  Short offline          Completed without error    00%         18
# 8  Short offline          Completed without error    00%          0
# 9  Short offline          Completed without error    00%          0

```

Again, the results of the tests are included in the output generated when **smartctl** is launched with the **-a** option; if one wants to focus only on logs, instead, he/she can use the **-l** option (**--log**) and specify what kind of logs should be displayed. To display only **error** logs, one would run:

```
$ sudo smartctl -l error /dev/sda
```

To include also **selftests** logs, instead:

```
$ sudo smartctl -l error -l selftest /dev/sda
```

When **smartctl** is launched with the **-a** option the **error**, **selftests** and **selective** logs are included in the output for ATA devices.

Conclusions

In this tutorial we saw how to install the **smartmontools** package on some of the most used Linux distributions and how to use the **smartctl** utility to check the status of S.M.A.R.T parameters on a storage device. We also saw what are the

possible tests we can run, and what are the differences between them. We also saw how to launch a test and how to the results. In the next article we will see how to schedule tests using the **smartd** daemon, and how to be notified with an email when an error is encountered. Stay tuned!